Run It, Then Route It: Execution-Based Routing Between Small and Large LLMs for Text-to-SQL

Andrew Shilman David Cho

San José State University
CMPE 252 Project Proposal

October 14, 2026

Abstract

Small LLMs are cheap and fast but often write wrong SQL, while large LLMs are more accurate but slower and more expensive. We propose a text-to-SQL router in which a small model (Qwen2.5-Coder-7B) always answers first and a small classifier predicts whether its answer is wrong, sending only flagged questions to a large model (Qwen2.5-Coder-32B). Because SQL can be run, crashes, empty results and disagreement between sampled queries all count as evidence. On BIRD Mini-Dev, the 7B scores 49.8% and the 32B 57.8%, but a perfect router would reach 63.8% while escalating only 14% of questions, so routing could improve accuracy as well as cost. On our training set, sample agreement alone gives ROC-AUC 0.784. We will compare our router with simple-rule, difficulty-label, random and oracle routers across the accuracy-vs-escalation curve, with paired significance tests, and measure what sampling costs.

1 Introduction

Text-to-SQL turns a question written in plain English into a SQL query that runs on a database, so people with no SQL or programming experience can still get answers from their data. Large language models do well on cross-domain benchmarks such as Spider [1], but questions on large, real databases with messy values are still hard for them, as BIRD shows [2].

BIRD has 12,751 question and SQL pairs over 95 databases. Each question comes with a short hint, which BIRD calls evidence, that gives outside knowledge needed to answer it [2]. We use the SQLite version of BIRD Mini-Dev [3, 4], which has 500 high-quality pairs from 11 databases, split by difficulty into 30% simple, 50% moderate and 20% challenging, along with its scripts for execution accuracy and Soft F1.

With the official Mini-Dev prompt and a free open-weight 7B coder model running on our laptop, we get 49.8% execution accuracy (EX), close to the 47.8% that the official baselines list for GPT-4. The 32B model from the same family [5] gets 57.8%, 8.0 points more, and McNemar's test shows the gap is not chance ($p < 0.001$). But the 32B is 4.2 times slower per question on our hardware. With paid models and APIs, it also costs a lot more [6].

Most questions don't need the large model. Of the 500 questions, both models answer 219 correctly and both get 181 wrong, so for 80% of them the 32B changes nothing. Switching to the 32B fixes 70 (14%) and breaks 30 (6%). A perfect router that escalates only the 70 it fixes would reach 63.8%, which is higher than always using the 32B. So routing can pay off in two ways, and we measure both: higher accuracy than always using the large model, and lower cost, since the small model runs locally and only the 32B calls are paid for.

SQL suits this kind of routing because, unlike a chatbot reply, a query can be run and its result checked. A query that crashes is wrong, a query that returns no rows is suspicious, and if several sampled queries for the same question return different results, the model is probably unsure [7, 8]. Malekpour et al. [9], who introduced LLM routing for text-to-SQL, chose a model before any SQL was written. They ruled out trying the small model first because, in their view, it is infeasible to tell whether generated SQL is accurate. We test whether running the small model's SQL and comparing its samples can make that call well enough to

decide when to route to the bigger model. Generating five samples takes about as long as running one on the 32B, so we also test whether the extra time is worth it.

Research question. Can cheap signals about a question, and about the small model's own answers, predict text-to-SQL failures well enough to route between a small and a large LLM?

Contributions.

1. A text-to-SQL router that predicts the small model's failures from execution results, token probabilities and agreement between sampled queries, all available before the large model is called.
2. A comparison of two training targets: "the small model is wrong" (A) vs. "the small model is wrong and the large model is right" (B), which marks exactly the questions worth escalating.
3. An evaluation on BIRD Mini-Dev with free models that run on a laptop, comparing our router with simple rules, BIRD's difficulty label, random choice and a perfect router at every share of questions sent to the large model, with significance tests.
4. Fixes for three problems in the official BIRD Mini-Dev downloads that affect anyone reproducing its results.

Section 2 defines the routing problem, Section 3 reviews related work, Section 4 describes the router, Section 5 lists the experiments, Section 6 covers the data, setup and preliminary results, Section 7 defines the metrics, and Section 8 gives the timeline.

2 Problem Statement

2.1 Setup

We start with a user question q, a database D and BIRD's evidence hint e. The small model S (Qwen2.5-Coder-7B) answers first with a SQL query $\hat{s}$. The router then decides whether to keep that answer or to escalate to the large model L (Qwen2.5-Coder-32B) and use its answer $\hat{l}$ instead.

We grade answers by execution accuracy (EX). Instead of comparing the SQL text, we run both the predicted SQL and the gold (correct) SQL g on the database. If they return the same rows, ignoring row order and duplicates, the answer is correct. Over N questions,

$$\mathrm{EX} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\big[\,\mathrm{set}\big(\mathrm{exec}(\hat{y}_i, D_i)\big) = \mathrm{set}\big(\mathrm{exec}(g_i, D_i)\big)\big]\,, \tag{1}$$

where $\hat{y}_i$ is the SQL the system returns for question i. A query that crashes or times out counts as wrong.

2.2 Routing rule

The classifier f turns a set of cheap signals into a score $p \in [0, 1]$, the estimated chance that the small model's answer should be escalated. The signals are the question, the database schema, the small model's SQL, the rows it returned, its token probabilities and a few extra sampled queries. If p is above a cutoff t, the question goes to the large model L:

$$p = f\big(\mathbf{x}(q, D, e, \hat{s}, \hat{s}^{(1)}, \ldots, \hat{s}^{(k)})\big), \qquad \hat{y} = \begin{cases} \hat{l} & \text{if } p > t \quad \text{(escalate to } L\text{)}, \\ \hat{s} & \text{otherwise,} \end{cases} \tag{2}$$

where $\hat{s}^{(1)}, \ldots, \hat{s}^{(k)}$ are k extra samples from the small model. We use $k = 5$ by default and test $k = 0$ to 5 in E3 and E5 (Section 5).

Moving t from 1 down to 0 sends anywhere from 0% to 100% of the questions to the large model, which gives a curve of accuracy against the share escalated. The router may only use information that exists before L is called. That rules out L's own output and BIRD's difficulty labels, which a real user question wouldn't have.

2.3 Training targets

We compare two training targets:

$$y^A = \mathbf{1}\big[\, S \text{ is wrong} \,\big], \qquad y^B = \mathbf{1}\big[\, S \text{ is wrong and } L \text{ is right} \,\big]. \tag{3}$$

Target A is the simpler one, and B is what routing needs (Table 1). A perfect A router knows which answers are wrong, but not which of them L can fix (only 70 of 251 on the test set), so at low escalation shares most of the questions it sends to L stay wrong. If B works better in practice, it means predicting which questions are fixable beats predicting which answers are wrong.

Table 1: The two training targets. The training set is the 1,034 BIRD dev questions not in Mini-Dev, and the test set is the 500 Mini-Dev questions. The last row shows how a perfect classifier for each target would route.

	A: Is the small model's answer wrong?	**B: Is this question worth escalating?**
Label is 1 when	S is wrong	S is wrong and L is right
Share of 1s, training	45.7% (473 of 1,034)	15.6% (161 of 1,034)
Share of 1s, test	50.2% (251 of 500)	14.0% (70 of 500)
Labels need	Runs of S only	Runs of S and of L on the training questions
Strength	Balanced classes, many examples, independent of which large model is used	Matches what routing needs, so it can reach high accuracy at low escalation
Risk	Escalates questions both models fail (181 of 500 on test), wasting large-model calls	Fewer positives (161 to learn from), and "fixable" may be hard to distinguish from "hopeless"
Perfect version: test EX at 14 / 20 / 30 / 50% escalated	53.7 / 55.4 / 58.2 / 63.7 (expected, with wrong answers escalated in random order)	63.8 / 63.8 / 63.8 / 63.8 (this is the oracle)

2.4 Goal

We want the highest EX at every escalation share. To be useful, the router has to beat the simple baselines (random routing, the difficulty label and one-line rules) by more than test-set noise (Section 7) and get as close to the oracle as it can. We also measure what the router costs in time and tokens, including whether the extra samples are worth the wait.

3 Related Work

Text-to-SQL benchmarks. Spider [1] is the standard cross-domain benchmark, but BIRD [2] is closer to real use, with large databases, messy values and questions that need outside knowledge. When BIRD came out, GPT-4 scored about 55% on it. We use its Mini-Dev subset because it is open, comes with official scoring scripts and difficulty labels, and our 7B gets about 50% on it, which gives the classifier a balanced mix of right and wrong answers.

LLM methods for text-to-SQL. Most text-to-SQL work tries to get more out of a single model, through prompt design (DAIL-SQL [10]), breaking the task into steps with self-correction (DIN-SQL [11]) or fine-tuning on synthetic data (OmniSQL [12]). We leave the generator alone and ask whether a cheap model's answer is good enough to keep.

Self-consistency. Self-consistency [7] samples several answers and takes a majority vote, and the authors also note that how much the samples agree works as an uncertainty estimate. We use it mainly in that second way. Our agreement feature counts how many of the small model's extra samples return the same rows as its main answer.

LLM cascades and routing. Routing between models to save cost has been studied before. FrugalGPT [6] cascades API calls, and RouteLLM [13] and Hybrid LLM [14] train routers that choose between a strong and a weak model. Mixture-of-Thoughts cascades [8] are closest to our setup, escalating when a weak model's sampled answers disagree. All of these were built for general text or math, where an answer can't be run. With SQL we can execute the query and look at what it returns.

Routing for text-to-SQL. Malekpour et al. [9] introduced the first LLM router for text-to-SQL. It reads the question and picks a model, such as Llama-3.1-8B or GPT-4o, before any SQL is written. They ruled out a cascade, arguing that it is infeasible to tell whether generated SQL is accurate before falling back to a stronger model, and that is the assumption we test. EllieSQL [15] also routes before generation, based on question complexity. The only system we found that escalates after running the SQL is Lee's agent [16]. Its 7B agent beat a 32B baseline, but its escalation layer did not improve results. Our E3 compares a router that decides before generation with one that runs the SQL first.

Detecting wrong SQL. Other work detects wrong SQL, but uses the signal to abstain, rerank or report confidence rather than to escalate. GradeSQL [17] reranks candidate queries with a reward model, and Somov and Tutubalina [18] abstain when token uncertainty is high. Entezari Maleki et al. [19] compare confidence methods on BIRD, where execution consistency gets ROC-AUC 0.66 to 0.76 depending on the model. Richardson [20] gets only 0.613 from execution agreement with GPT-4o-mini, while agreement gives 0.784 for our 7B on our training set. The setups differ (8 vs. 5 samples, different questions and models), but the usefulness of agreement may depend on the model and setup.

Gap we fill. Most text-to-SQL routers choose a model before any SQL is written [9, 15], and work that checks SQL after the fact uses the result to abstain, rerank or report confidence [17–21]. As far as we know, no one has escalated BIRD questions from a cheap model to an expensive one based on how well the cheap model's executed samples agree. We propose to build this and compare two ways to train it: predicting whether the small model is wrong, and predicting whether the large model can fix it.

4 Proposed Solution

4.1 Overview

Figure 1 shows the pipeline. The 7B answers the question at temperature 0, and we also draw k extra samples (5 by default) at temperature 0.7. We run all of this SQL on the database, compute features from the results and pass them to the classifier. If its risk score p is above the cutoff t, we drop the 7B's answer, call the 32B and use its answer. The classifier is the only thing we train. Both LLMs are used as they are, with no fine-tuning.

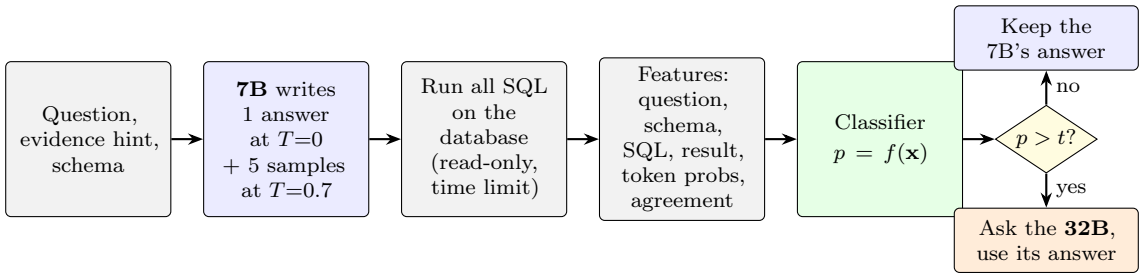


Figure 1: The router pipeline. By default we keep the 7B's temperature-0 answer and use the 5 samples to measure agreement (E4 also tests keeping the samples' majority answer). All features are computed before the 32B is called.

4.2 Models

All three models come from one open-weight family [5], so only the size changes. The 32B is the largest Qwen2.5-Coder model and still fits on our hardware.

We picked the small model closest to 50% EX so the classifier sees a balanced mix of right and wrong answers. The 7B gets 49.8%, while with the 1.5B (17.8%) a classifier that always says "wrong" would already be 82% accurate. The 32B is free, local and 8.0 points better than the 7B, so we saw no reason to pay for an API model.

Table 2: Models. All three are Qwen2.5-Coder Instruct models [5] served locally through Ollama [22] in its default 4-bit (Q4_K_M) versions. EX is on Mini-Dev (500 questions). Seconds and output tokens are per question on the test set.

	Small model	**Large model**	**Tried, dropped**
Model	Qwen2.5-Coder-7B-Instruct	Qwen2.5-Coder-32B-Instruct	Qwen2.5-Coder-1.5B-Instruct
Parameters	7.6B	32.8B	1.5B
Mini-Dev EX (%)	49.80	57.80	17.80
Seconds per question	0.99	4.15	2.55
Output tokens per reply	56	59	330

4.3 Prompt and generation settings

The official prompt keeps our scores comparable with the published Mini-Dev baselines.

- **Prompt:** the official Mini-Dev prompt [3]: the database's `CREATE TABLE` statements (no example rows), the question, the evidence hint as "External Knowledge", a "think step by step" line and an instruction to return only SQL.
- **Sampling:** temperature 0 for the answer, with each token's log-probability from the same call, temperature 0.7 for up to 5 samples, sample s with seed s.
- **SQL extraction:** the code block if there is one, from the first `SELECT` or `WITH` to the first semicolon.

4.4 Which answer is kept, and what the samples are for

By default the router keeps the temperature-0 answer. It is repeatable and a little more accurate than the average sample (54.26% vs. 53.4% on the training set). The extra samples are there to measure agreement, which we judge only by execution results: two differently written queries agree if they return the same rows. For example, on training question 308 the 7B counts chlorine atoms instead of bonds, and only one of its five samples agrees with it. In E4 we also test letting the samples vote and keeping the majority answer, which scores 58.22% on training.

4.5 Features

Table 3 lists the planned features. Any selection among them will use only the training data. We leave out the difficulty label and anything from the 32B. We also leave out database identity, except in one test of its effect, because it wouldn't carry over to a new database. Token probabilities come free with the 7B's first call, so they are the obvious cheaper alternative to drawing extra samples.

4.6 Classifiers and routing

For each target we train logistic regression, random forest and gradient boosting [23], tuned by 5-fold cross-validation on the training questions over 3 seeds, and check unseen databases with leave-one-database-out cross-validation. The chosen models then score the 500 test questions once, and routing is simulated from saved answers. Since the training set is easier (Section 6.2), we report fixed escalation shares (20%, 30%, 50%), taking each cutoff from the test scores, which needs no labels. The classifier also has to beat two one-line rules: escalate if the 7B's SQL crashes, or if at most j samples agree with it (Table 8).

Table 3: Planned features. All of them can be computed before the large model is called.

Group	Features	Cost
Question	Word count, numbers present, keywords like "percentage", "ratio", "average", "highest", length of the evidence hint.	Free
Schema	Total tables and columns, mentions of specific tables or columns in the question.	Free
Small model's SQL	Did it crash?, was the result empty?, row count, number of JOINs and subqueries, SQL length, reply token count.	1 small-model call (already made)
Token confidence	Mean and lowest token log-probability of the temp-0 SQL, count of low-probability tokens.	Free (part of the temp-0 call)
Self-agreement	How many of the k samples returned the same rows as the temp-0 answer, number of distinct results, crash count, identical text count.	k extra small-model calls ($k \leq 5$)
Database (impact test only)	Which of the 11 databases the question is from.	Free

5 Proposed List of Experiments

Table 4 lists our experiments. E1 is already done (Section 6.5), and E2 to E5 are planned, with E4 as the main result: the accuracy-vs-escalation curve on the test set.

Table 4: Proposed experiments. A is the "small model wrong" target and B the "worth escalating" target. k is the number of temperature-0.7 samples.

ID	Question	Method	Metrics	Status
E1	How good is each model alone?	1.5B, 7B and 32B zero-shot on Mini-Dev	EX and Soft F1 by difficulty, plus a McNemar test of 7B vs. 32B	Done
E2	Which classifier predicts failures best?	Logistic regression, random forest and gradient boosting for targets A and B, with class weights, each tuned by 5-fold CV on the 1,034 training questions over 3 seeds, with leave-one-database-out CV as a transfer check	ROC-AUC, PR-AUC and F1 on the positive class, as mean ± std	Planned
E3	Which features matter, and how many samples are worth it?	Ablation for A and B in steps: question and schema only (a router that decides before generation), then add the 7B's SQL, execution result and token probabilities (no extra calls), then agreement from $k = 1$ to 5 samples. Database identity on and off	ROC-AUC and PR-AUC per step and per k	Planned
E4	**Main result:** accuracy vs. escalation	Train the chosen classifiers on all training data, score Mini-Dev once and sweep the cutoff. Compare with always-small, always-large, random, difficulty-label, two simple rules (the 7B's SQL crashed, or at most j samples agree) and the oracle. Also A vs. B, and keeping the temperature-0 answer vs. the majority answer	EX at 20/30/50% with paired-bootstrap 95% CIs, area under the curve, and the share needed to match always-large	Planned
E5	What does each router cost?	Saved timings and tokens for $k = 0$ to 5, SQL execution time, sequential vs. batched sampling, and a dollar estimate where only large-model calls are paid	Seconds, tokens and dollars per question	Planned

Hypotheses.

H1: Execution-based features predict 7B failures well above chance (agreement alone: training ROC-AUC 0.784).

H2: Our router beats random and difficulty-label routing at 20, 30 and 50% escalation, with paired-bootstrap 95% CIs excluding zero (the difficulty label's own +0.8 points at 20.4% does not).

H3: The trained classifier beats the best simple agreement rule at equal escalation (on training that rule gets 62.96% at 25.1%, random 56.81%).

H4: Agreement adds to the single answer's features, token probabilities included, and most of it comes from the first 2–3 samples (training ROC-AUC 0.701 with 1 sample, 0.765 with 3, 0.784 with 5).

H5: Target B beats target A at low escalation shares, if it can be learned from 161 positives.

H6: A router matches always-large (57.8%) while escalating at most half of the questions (on training the agreement rule does: 64.70% at 47.9% vs. 64.41%).

Rules for every experiment.

- Never train, tune or choose features on Mini-Dev. It is scored once, at the end. Preliminary numbers that use the 7B's outputs are training-set numbers.
- BIRD's difficulty label is a baseline router only, never a feature, and nothing from the large model is a feature either.
- All classifier numbers are mean ± std over 3 seeds. Differences between routers on the test set come with paired-bootstrap confidence intervals, and EX and Soft F1 come from the official Mini-Dev scripts.

6 Experiment Setup and Dataset Details

6.1 Datasets and licenses

All of our data is open source: the BIRD dev set [2] and the Mini-Dev questions and databases [3, 4]. Each question has an ID, its database, the question text, an evidence hint (missing for 146 training questions), the gold SQL and a difficulty label (simple, moderate or challenging).

6.2 Training and test split

All 500 Mini-Dev questions also appear in the 1,534-question BIRD dev set, so we remove them from the training pool by question_id. Matching by text would miss some, since 4 of them were reworded. That leaves 1,034 training questions on the same 11 databases as the test set.

The two sets differ in opposite directions (Table 5). The training mix is much easier, with only 43 challenging questions, yet on simple and moderate questions the 7B does worse on it. Missing hints explain part of this: without a hint the 7B gets 43.2% on training, against 56.1% with one. We handle this by reporting results at fixed escalation shares and by including hint features.

Table 5: Training and test sets: difficulty mix, basic statistics, and the 7B's accuracy by difficulty.

	Training	Test (Mini-Dev)
Simple	777 (75%)	148 (30%)
Moderate	214 (21%)	250 (50%)
Challenging	43 (4%)	102 (20%)
Total questions	**1,034**	**500**
Evidence hint length, words (mean)	12.2	19.4
Questions with no evidence hint	146 (14.1%)	2 (0.4%)
Gold SQL with at least one JOIN	733 (71%)	407 (81%)
Gold SQL with a nested SELECT (subquery, UNION, etc.)	61 (6%)	64 (13%)
7B EX (%): simple / moderate / challenging	59.07 / 39.72 / 39.53	66.22 / 47.20 / 32.35
7B EX (%): total	54.26	49.80

6.3 Data problems found and fixed

We found three problems in the official downloads. First, the Mini-Dev gold SQL file matches its own questions on only 494 of 500 lines, so we rebuild the test files from the maintained Hugging Face copy [4].

Second, 5 of the 11 databases have different rows in the two downloads, which changes the correct answer for 23 test and 82 training questions, so each question set uses its own databases. Third, the official 30 s limit covers both the predicted and the gold SQL, and two gold queries need more than that on their own (one takes about 100 s), so even the gold SQL scores only 99.60%. At 400 s it scores exactly 100.00%.

6.4 Hardware, software and code

Everything runs on one Apple M5 Max laptop with 64 GB of memory. The models are served by Ollama [22] on the GPU, our scripts are in Python, and the classifiers will use scikit-learn [23].

We reuse the Mini-Dev repository's prompt code and its EX and Soft F1 scorers without changes [3]. Our own scripts [24] fix the data, build the training file, query the models while caching every reply with its timing and token log-probabilities, and mark every saved answer right or wrong. The checker reproduces the official totals exactly for all three models.

6.5 Preliminary results

In E1 (Table 6) the 32B beats the 7B at every difficulty, and the gap is largest on challenging questions. Crashes alone are a weak signal: only 57 of the 7B's 251 wrong answers crashed, so a crash rule misses most mistakes. A single EX score on 500 questions has a margin of error of about ±4.4 points, so we compare routers with paired tests.

Table 6: E1: each model alone on Mini-Dev (scored at 400 s). EX by difficulty, total EX, Soft F1 and how many of each model's 500 answers crashed.

Model	Simple (148)	Moderate (250)	Challenging (102)	EX (500)	Soft F1	Crashed
Qwen2.5-Coder-1.5B	29.73	15.20	6.86	17.80	19.62	277 (55%)
Qwen2.5-Coder-7B	66.22	47.20	32.35	49.80	52.68	57 (11.4%)
Qwen2.5-Coder-32B	72.30	54.80	44.12	57.80	61.62	17 (3.4%)

Routing by BIRD's difficulty label, escalating all 102 challenging questions, gives 52.2%. Random routing at the same share gives 51.4%, and the 95% confidence interval of the difference (−0.9 to +2.5) includes zero (Figure 2a). Hard questions are where the 7B fails, but they are also where the 32B fails, so knowing a question is hard says little about whether escalating will fix it. A perfect router would reach 63.8% (Table 7).

Table 7: 7B vs. 32B, question by question. The 32B fixes only 70 of the 7B's 251 wrong test answers (27.9%) and gets 30 of its right answers wrong.

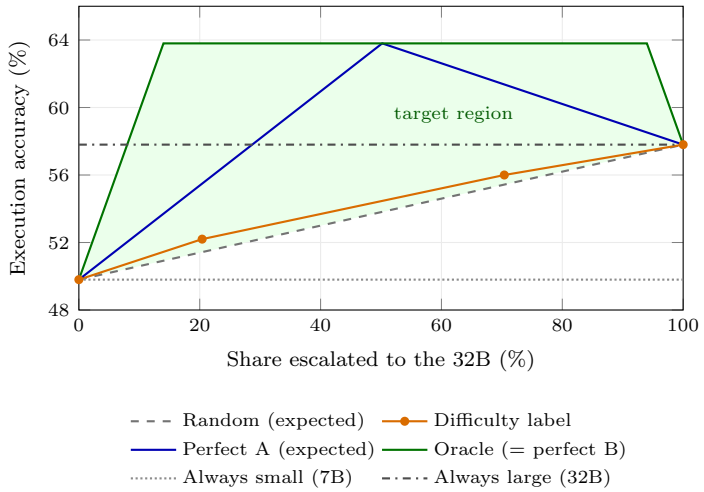
	Test (500)		Training (1,034)	
	32B right	32B wrong	32B right	32B wrong
7B right	219	30	505	56
7B wrong	70	181	161	312
EX: always small / always large / oracle	49.80 / 57.80 / 63.80		54.26 / 64.41 / 69.83	

On the training set, a temperature-0.7 sample is on average slightly less accurate than the temperature-0 answer (53.4% vs. 54.26%), but the majority answer of the 5 samples gets 58.22%. That is why E4 also tests keeping the majority answer.

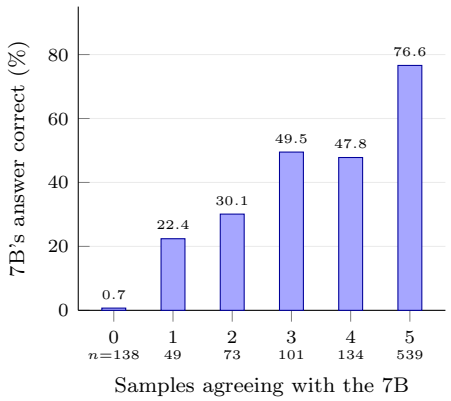
Agreement is our strongest early signal (Figure 2b, training set). Counting how many samples match the 7B's main answer gives ROC-AUC 0.784 for target A and 0.778 for target B. The 7B's mean token log-probability, which costs nothing extra, gives 0.731. As a one-line rule (Table 8), agreement beats random routing by 6.2 points at 25% escalation and slightly beats always-large at 48% escalation (64.70% vs. 64.41%). This rule is the bar our trained classifier has to clear.

Table 8: Simple rule-based routers tested on the training set (1,034 questions, both models at temperature 0). "Agree" counts how many of the 5 samples return the same rows as the 7B's main answer. The test set is not used until E4.

Router (escalate when . . .)	Share escalated	EX (%)	Random, same share
Never (always small)	0%	54.26	–
Difficulty is challenging	4.2%	54.93	54.68
Difficulty is challenging or moderate	24.9%	56.96	56.78
The 7B's SQL crashed	11.1%	59.67	55.38
No sample agrees	13.3%	59.86	55.61
At most 2 of 5 agree	25.1%	**62.96**	56.81
At most 4 of 5 agree	47.9%	64.70	59.12
Always (always large)	100%	64.41	–
Oracle (only questions the 32B fixes)	15.6%	69.83	55.84



(a) Reference routing curves on the test set.



(b) Agreement vs. correctness on the training set.

Figure 2: Preliminary results. (a) Our router's curve should land in the shaded region between random and the oracle. Rule-based routers are evaluated only on the training set (Table 8). Perfect A escalates only wrong 7B answers, in a random order, the oracle escalates the 70 questions the 32B can fix first, and the 30 it would break last. (b) The more samples agree with the 7B's temperature-0 answer, the more often that answer is correct, with a small dip from 3 to 4.

7 Proposed Evaluation Metrics

We measure four things: how accurate the SQL is, how well the classifier predicts failures, how well the router trades accuracy against escalation, and what it costs. Routers are compared with paired significance tests.

7.1 Text-to-SQL accuracy

Our main metric is execution accuracy (EX, Equation (1)), the share of questions where the predicted SQL returns the same set of rows as the gold SQL. Row order and duplicate rows don't matter, but the columns do, and a query that crashes or passes the 400 s limit counts as wrong. We also report Soft F1, the second metric in the Mini-Dev scripts (still marked beta there). It matches values row by row between the predicted and gold results and computes $F_1 = 2PR/(P + R)$, so an answer with swapped columns or a few missing values still gets partial credit. Both come from the official scripts [3].

7.2 Classifier quality (E2, E3)

ROC-AUC is the chance that a randomly chosen positive (for target A, a wrong 7B answer) gets a higher risk score than a randomly chosen negative. PR-AUC is the area under the precision-recall curve, and it matters most for target B, where only 15.6% of training questions are positive. We also report precision, recall and F1 on the positive class, at the cutoff that maximizes F1 and at fixed escalation shares. All of these come from 5-fold cross-validation on the training set, averaged over 3 seeds. Leave-one-database-out cross-validation then estimates how well the classifier carries over to a database it has never seen.

7.3 Routing quality (E4)

The escalation share r is the fraction of questions sent to the large model. We report EX at $r = 20\%$, 30% and 50%, and the share needed to match always-large (57.8%). We also plot EX against every share from 0% to 100% and report the area under this routing curve, $\int_0^1 \mathrm{EX}(r)\,dr$. For reference, on the test set this area is 53.80 for random routing, 54.30 for the difficulty label, 58.79 for perfect A and 62.64 for the oracle.

Two routers usually give the same answer on most questions, so their scores are strongly linked. Comparing their separate ±4.4-point margins would hide real differences, so we test the per-question differences directly, with a paired bootstrap (10,000 resamples) [25] and McNemar's exact test [26]. These tell us whether a router's gain over random, the difficulty label and the rules is larger than chance.

7.4 Cost (E5)

Cost includes time as well as money. For each router we report seconds and tokens per question, and a dollar estimate that assumes the small model runs locally and only large-model calls are paid. The time and dollar cost per question are

$$T_{\text{router}}(r,k) = T_S + k \cdot T_{\text{sample}} + T_{\text{exec}} + r \cdot T_L, \qquad \$_{\text{router}}(r) = r \cdot \$_L, \tag{4}$$

where T_S is the 7B's answer, T_{sample} one extra sample, T_{exec} running all the SQL, T_L one 32B call and $\$_L$ the price of one 32B call.

On our test set, the 7B takes 0.99 s, one sample 0.91 s, the 32B 4.15 s, and running the SQL 0.22 s. With $k = 5$ samples drawn one after another, the router spends at least 5.8 s per question, which is slower than calling the 32B every time. So the samples have to pay for themselves in accuracy, or we need fewer of them, or we need to batch them, which E5 tests. Without samples, the router is faster than always-large as long as it escalates less than about 75% of the questions.

8 Project Execution Timeline

8.1 Completed so far

The groundwork is already done. In the first week we set up the environment, fixed the data problems, ran all three baselines, chose the models and the two targets, and finished every model run the project needs, so every training question already has labels A and B. The training-set runs alone took about 2 h 43 min of model time. Since then we have also measured Soft F1, agreement, the simple-rule routers and token log-probabilities.

8.2 Plan

Table 9 shows the plan week by week, ending with the final report, which is due on Wednesday 16 December. The Owner column shows who leads each part, and we both write the final report.

Table 9: Weekly plan, owners and outputs.

Week	Dates	Work	Owner	Output
1	5–11 Oct	Setup, data fixes, baselines (E1), model decisions, training data and labels, plus agreement with the temperature-0 answer, simple-rule routers and token log-probabilities	Andrew	Done
2	12–18 Oct	Finish and submit the proposal (due Wed 14 Oct), then start the feature script	Both	Proposal
3	19–25 Oct	Feature table for training and test, with sanity checks	David	Feature table
4	26 Oct–1 Nov	E2: three classifiers × targets A and B, 5-fold CV over 3 seeds, and leave-one-database-out CV	David	E2 table
5	2–8 Nov	E3: feature ablation, including $k = 1$ to 5 samples, token probabilities and database identity, then choose the final classifiers using CV only	David	E3 table and final models
6	9–15 Nov	E4: score Mini-Dev once, routing curves against all reference and rule routers, A vs. B, and temperature-0 vs. majority answer, with bootstrap CIs	Andrew	Main result
7	16–22 Nov	E5: cost and latency, including batched sampling (Andrew), and error analysis (David)	Both	E5 table, analysis
8	23–29 Nov	Start the final report	Both	Report draft
9	30 Nov–6 Dec	Write the final report (12 to 30 pages), make the repository public, document each member's contributions	Both	Full draft
10	7–16 Dec	Buffer, final review and submission (due Wed 16 Dec)	Both	Final report

References

[1] Tao Yu, Rui Zhang, Kai Yang, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[2] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, pages 42330–42357, 2023.

[3] Xiaolong Li, Jinyang Li, Ge Qu, Binyuan Hui, Reynold Cheng, and Chenhao Ma. BIRD-SQL Mini-Dev. GitHub repository, commit abd11b6 (5 September 2026), 2026. URL https://github.com/bird-bench/mini_dev. License: CC BY-SA 4.0.

[4] BIRD Team. BIRD Mini-Dev dataset (birdsql/bird_mini_dev). Hugging Face dataset, mini_dev_sqlite split, last modified 18 January 2026, 2026. URL https://huggingface.co/datasets/birdsql/bird_mini_dev. License: CC BY-SA 4.0.

[5] Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. Model weights (Apache 2.0) on Hugging Face: Qwen/Qwen2.5-Coder-7B-Instruct and Qwen/Qwen2.5-Coder-32B-Instruct.

[6] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[8] Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Mohammadhossein Malekpour, Nour Shaheen, Foutse Khomh, and Amine Mhedhbi. Towards optimizing SQL generation via LLM routing. In *NeurIPS 2024 Table Representation Learning Workshop*, 2024. arXiv:2411.04319.

[10] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

[11] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[12] Haoyang Li, Shang Wu, Xiaokang Zhang, et al. OmniSQL: Synthesizing high-quality text-to-SQL data at scale. *Proceedings of the VLDB Endowment*, 18(11):4695–4709, 2025.

[13] Isaac Ong, Amjad Almahairi, Vincent Wu, et al. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Yizhang Zhu, Runzhi Jiang, Boyan Li, Nan Tang, and Yuyu Luo. EllieSQL: Cost-efficient text-to-SQL with complexity-aware routing. In *Conference on Language Modeling (COLM)*, 2025.

[16] Morris Lee. Business truth, not SQL accuracy: A rule-gated 7B analytics agent outperforms a direct-prompted 32B baseline. *arXiv preprint arXiv:2608.09254*, 2026.

[17] Mattia Tritto, Giuseppe Farano, Dario Di Palma, Gaetano Rossiello, Fedelucio Narducci, Dharmashankar Subramanian, and Tommaso Di Noia. GradeSQL: Test-time inference with outcome reward models for text-to-SQL generation from large language models. *arXiv preprint arXiv:2509.01308*, 2025.

[18] Oleg Somov and Elena Tutubalina. Confidence estimation for error detection in text-to-SQL systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25137–25145, 2025.

[19] Sepideh Entezari Maleki, Mohammadreza Pourreza, and Davood Rafiei. Confidence estimation for text-to-SQL in large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 32474–32482, 2026.

[20] Robert Richardson. What predicts correctness in text-to-SQL? A selective-prediction study. *arXiv preprint arXiv:2607.06799*, 2026.

[21] Mincheol Daniel Song, Joshua Ward, Jake Jung, and Guang Cheng. Trajectory-derived confidence for reliable, resource-aware clinical text-to-SQL agents. In *NeurIPS 2026 Workshop on Resource-Aware Agentic AI*, 2026. arXiv:2610.04156.

[22] Ollama. Ollama. GitHub repository, version 0.35.1, 2026. URL https://github.com/ollama/ollama. MIT License.

[23] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.

[24] Andrew Shilman and David Cho. text2sql-router. GitHub repository, 2026. URL https://github.com/Ashilman25/text2sql-router.

[25] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[26] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.